

Constructeurs

- Si, et seulement si, aucun constructeur n'est spécifié pour une classe, le compilateur en définit implicitement un, sans paramètre :
 - Le **constructeur par défaut**, qui ne fait (*presque*) rien
 - Il est **public**
- Même effet qu'un constructeur défini par l'utilisateur avec un corps vide et une liste d'initialisation vide
- L'implémentation de ce constructeur par défaut invoque le constructeur sans paramètre des classes parentes si elles existent.
- Et des membres non statiques de cette classe
- Attention**: lors d'une initialisation par défaut (`MyClass obj;`), les types fondamentaux (`int`, `float`, `pointeurs`, etc.) ne sont pas initialisés et leur valeur est indéterminée. En revanche, avec une initialisation par accolades (`MyClass obj{}`), ils sont initialisés à zéro
 - > préférer construire vos objets avec initialisation e.g.: `MyClass obj{}`;

C++ 22

Opérateurs particuliers (3)

- Opérateur de conversion de type (*transtypage*), `static_cast<Type>(obj)`, surchargé par une méthode `Classe::operator Type()`.
 - `Vector::operator double() const` {
 return `sqrt(x * x + y * y)`;
}
 - `Vector v(3, 3);`
`double d = v;` // invoque `v.operator double()`;
- Conversion d'un `String` en un `const char*` (pour utiliser des bibliothèques):
`class String`
 {
 char* value;
 public:
 explicit `operator const char*() const` { return value; }
 };
 String s{"foo"};

 std::println("{} ", static_cast<const char*>(s));

C++ 52

Bonnes pratiques - révisée (post C++ 11)

- Une classe `T` est dite sous **forme canonique** si elle offre les **trois cinq** méthodes suivantes:
`class T` {
 T(const T&); // Constructeur de copie
 T(Vector&&); // Constructeur de déplacement
 ~T(); // Destructeur
 T& operator=(const T&); // Opérateur d'affectation
 T& operator=(T&&); // Opérateur d'affectation par déplacement
};
- En général, s'il existe une allocation de ressources dans un constructeur (eg `new/new[]/fopen`, etc)
 - Désallocation dans le destructeur (`delete/delete[]/fclose`, etc.),
 - Définition du constructeur de copie avec allocation,
 - Définition du constructeur de déplacement avec appropriation de la ressource,
 - Surcharge de l'opérateur = avec désallocation (courant) et réallocation,
 - Surcharge de l'opérateur = (move) avec désallocation et appropriation de la ressource.
- "Règle des trois-cinq"**:
 - Quand on implémente *une* de ces cinq méthodes, on doit également implémenter *les quatre autres*.

C++ 23

Passage de paramètres - révisé

- Non modification des données originales
 - Par référence constante (préférable): `void f(const string& s);`
 - Par pointeur constant (peut être `nullptr`): `void f(const string* ptr);`
 - Par valeur (copie / ctor de copie / ctor de dépl.): `void f(string s);`
 - Si l'objet passé est un *rvalue* (par exemple, le résultat d'une expression ou un objet temporaire), le constructeur de déplacement est préféré si disponible, permettant ainsi une optimisation en transférant les ressources (*).
- Modification autorisée
 - Par référence: `void f(string& s);`
 - Par pointeur: `void f(string* ptr); void f(string s);`
- Transfert de la donnée:
 - Par déplacement (ctor de dépl.): `void f(string&& s);`

C++ * Cette opération peut parfois être optimisée par le compilateur, évitant ainsi la construction et la destruction inutiles. 25

Types résultat - révisé

- Valeur:
 - `x f() { /* ... */ }`
`x x = f();` // le résultat de `f` est copié (cons. de copie pour les objets) // ou déplacé si `x` supporte la sémantique de déplacement (cons. de déplacement).
 - `x = f();` // copie (ou déplacement si supporté) de la valeur de la variable référencée rendue (affectation par déplacement) (*).
- Référence r-value:
 - `X&& f() { /* ... */ }` // généralement **à éviter** // sauf si vous savez ce que vous faites // car il est problématique de déplacer un objet // qui va être détruit à la fin de la fonction `f`).
 - Exception: nécessaire (et valide) pour implémenter `std::move` car l'argument passé en paramètre est retourné (donc pas détruit).
Implémentation (simplifiée) de `std::move`:
`template<typename T>`
`T&& move(T& arg) {`
 return `static_cast<T&&>(arg);`
}

C++ * Cette opération peut parfois être optimisée par le compilateur, évitant ainsi la construction et la destruction inutiles. 26

Sémantique de déplacement

```
// a est passé par référence lvalue
void f(const string& a) { cout << a << " : f(&)\n"; }

// a est passé par référence rvalue
void f(string&& a) { cout << a << " : f(&&)\n"; }

string fn() {return "abc"s;}

int main() {
  std::string a = "abc"s;
  f(a); // abc : f(&)
  f("abc"s); // abc : f(&&)
  f(std::move(a)); // abc : f(&&)
  f(fn()); // abc : f(&&)
  f(a); // <indéterminé (mais légal...)> : f(&)
}
```

C++ 32

Sémantique de déplacement

```
void f(const string& a) { cout << "f(&)\n"; }
void f(string&& a) { cout << "f(&&)\n"; }
void g(const string& a) { cout << "g(&)" - " : f(a); }
void g(string&& a) { cout << "g(&&)" - " : f(std::move(a)); }

int main() {
  string a = "hello"s;
  g(a); // g(&) - f(&)
  g(std::move(a)); // g(&&) - f(&&)
}
```

C++ 36

Sémantique de déplacement

Le constructeur de déplacement *peut* être automatiquement appelé dans les situations suivantes (si le constructeur de déplacement existe) (*):

- Retour par valeur**:
 - lors du retour de *rvalue*;
 - retour d'un objet local (*lvalue*), s'il peut être traité comme une *rvalue* par le compilateur
- Passage en paramètre par valeur**: pour les objets *rvalue*
- Lorsqu'une **exception** est lancée, l'objet "lancé" peut être déplacé.
- Initialisation avec un Temporaire**: Lorsqu'un objet est initialisé avec un objet temporaire (*rvalue*). Par exemple, `MaClasse obj = MaClasse();`.
- Utilisation explicite de `std::move`**.
- Échange d'objets** avec `std::swap`.
- Placement ou redimensionnement dans les conteneurs de la STL** les éléments sont déplacés vers le nouvel emplacement, si l'objet est une *rvalue*. Par exemple: `vect.push_back(MaClass());`

C++ * L'appel du constructeur de déplacement (ou de copie) peut parfois être optimisée par le compilateur, évitant ainsi la construction et la destruction inutiles. 37

Références universelles

```
void f(string& a) { cout << "f(&)\n"; }
void f(string&& a) { cout << "f(&&)\n"; }

// a: référence universelle
template<typename T>
void g(T&& a) {
    // std::forward préserve le caractère lvalue ou rvalue de 'a'
    cout << "g(universelle) - ";
    f(std::forward<T>(a));
}

int main() {
    string a = "hello"s;

    // a est une lvalue, std::forward<T>(a) agit comme une lvalue
    g(a); // g(universelle) - f(&)

    // a est converti en rvalue, std::forward<T>(a) agit comme une rvalue
    g(std::move(a)); // g(universelle) - f(&&)
}
```

C++ 39

Propriétés statiques

- Une **propriété statique** (ou *de classe*) appartient à la classe (et non à l'instance) et est disponible même s'il n'existe aucune instance de la classe.
 - Accès sur la classe **Classe::propriété**, sur un objet **objet.propriété**.
- Attribut statique** (constantes générales, compteur d'instances, etc.):
 - Possède la **même valeur** pour tous les objets de la classe.
 - Doit *généralement* être initialisé en dehors de la déclaration de la classe, sauf si:
 - Le type est **const**
 - Le type est marqué "inline" (C++17)
 - `inline static double b = 42.0;`
 - Le type est intégral, eg int, char, ... (C++20)
 - `static int x = 42;`
- Méthode statique** (méthode utilitaire, etc.):
 - Ne peut accéder aux propriétés non statiques (pas de pointeur `this`).
 - Considérer une fonction libre si la fonction est indépendante du contexte d'une classe.

C++ 4

Héritage: constructeurs et destructeur

- Pour initialiser une instance d'une sous-classe:
 - D'abord initialiser les attributs hérités,
 - Puis initialiser les attributs spécifiques.
- Constructeurs**: appel au(x) constructeur(s) de(s) super-classe(s),
 - Par défaut invocation du constructeur sans paramètre de la super-classe (qui doit exister).
 - Le constructeur de copie (resp. déplacement) par défaut invoque le constructeur de copie (resp. déplacement) de la super-classe.
 - Invocation d'un constructeur spécifique dans la liste d'initialisation:

```
class Vector3D : public Vector2D {
    int z;
public:
    Vector3D(int x, int y, int z) : Vector2D(x, y), z{z} { }
```
- Destructeur**: le destructeur d'une sous-classe invoque automatiquement le(s) destructeur(s) de sa (ses) super-classe(s).

C++ 10

Liaison dynamique (2)

- Une méthode redéfinie peut posséder un type de retour plus spécialisé que celui de la méthode originale si c'est un pointeur ou une référence:
-> **covariance du type résultat** (partielle).
- Les constructeurs ne peuvent être virtuels.
 - Pourtant le constructeur de copie mériterait d'être virtuel.
-> Définir une méthode virtuelle, utilisant le constructeur de copie.

```
class A {
    [[nodiscard]]
    virtual A* clone() const { return new A(*this); }
};
class B : public A {
    [[nodiscard]]
    B* clone() override const { return new B(*this); }
};
A* a = new B();
A* b = a->clone();
delete a; delete b; // Attention !
```

C++ 14

transtypage - en résumé

Opérateur	Fonction	À utiliser pour...
<code>dynamic_cast<T></code>	Convertit les types dans les hiérarchies de classes	Pointeurs et des références de classes dans une hiérarchie avec héritage. À préférer pour les types polymorphes (avec méthodes virtuelles). Note : Peut être coûteux en performance.
<code>static_cast<T></code>	Convertit les types de manière statique	- Pointeurs et des références de classes dans une hiérarchie avec héritage (attention : sans vérification à l'exécution). Note : plus performant que <code>dynamic_cast</code> . - <code>void*</code> en n'importe quel type pointeur. - entre types fondamentaux (<code>int</code> , <code>float</code> , ...) - types énumérés et <code>int</code> - appeler explicitement un ctor prenant un seul argument ou un opérateur de conversion. - convertir des types en références <code>rvalue</code> , - peut convertir n'importe quel type en <code>void</code>
<code>reinterpret_cast<T></code>	Convertit n'importe quel type de pointeur en un autre type de pointeur	- Convertit tout type de pointeur en un autre type de pointeur, même entre des classes non liées. Le résultat de l'opération est une copie binaire de la valeur d'un pointeur vers l'autre. Toutes les conversions de pointeurs sont autorisées : ni le contenu pointé ni le type de pointeur lui-même n'est vérifié. - Convertit des pointeurs en types entiers ou inversement. Le format dans lequel cette valeur entière représente un pointeur est spécifique à la plateforme. Note : Très spécifique et potentiellement dangereux. Peut facilement conduire à un comportement indéfini si utilisé incorrectement.
<code>const_cast<T></code>	Modifie ou enlève le qualificatif <code>const</code> .	Pour modifier le <code>const</code> d'un objet. À utiliser avec prudence. Note : Modifier un objet initialement déclaré <code>const</code> conduit à un comportement indéfini.
Conversion de style C (fonctionnelle) <code>T x = T(y);</code>		Note : Moins sûr et moins clair que le transtypage C++ explicite. Peut mener à des conversions non intentionnelles ou dangereuses, pour les types fondamentaux similaire à <code>static_cast</code> , pour les types classe similaire à <code>reinterpret_cast</code> .
Conversion de style C (cast)		

const_cast: exemple

```
// Une API externe
void legacy_API(char* data);

void process_data(const char* data_const) {
    // Utilisation de const_cast pour retirer la constance
    // car nous savons que legacy_API ne modifiera pas les données

    char* data = const_cast<char*>(data_const);
    legacy_API(data);
}
```

transtypage: énumérations

- Le type sous-jacent représentant les valeurs d'un **enum** peut être spécifié par le programmeur. En l'absence de spécification, le compilateur choisit un type entier suffisamment grand pour stocker toutes les valeurs de l'énumérateur (*généralement* `int`). Le type doit être un entier (`short`, `int`, `unsigned int`, `long`, etc.).
- `std::to_underlying` (C++23) permet d'obtenir la valeur sous-jacente de l'**enum**. Cette fonction est utile pour convertir une valeur d'énumération en son type sous-jacent sans avoir recours à une conversion de type explicite.
- `std::underlying_type_t<T>` (C++23) détermine le type sous-jacent de l'**enum**.

```
enum class Couleur : uint32_t { Rouge=1, Vert=2, Bleu=3 };

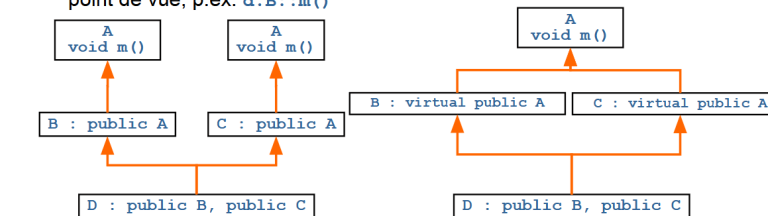
int main() {
    Couleur c = Couleur::Vert;
    // dans ce cas TypeSousJacent est uint32_t
    using TypeSousJacent = std::underlying_type_t<Couleur>;

    TypeSousJacent val = std::to_underlying(c);
    cout << "Valeur : " << val << endl;
    cout << "Type : " << typeid(TypeSousJacent).name() << endl;
}
```

C++ 26

Héritage multiple (MI) en losange

- Classe de base (**A**) non virtuelle.
 - Chaque sous-classe hérite de sa *propre* version de **A**.
 - Deux versions de **A** pour **D**. et donc de ses attributs!
 - `D d; d.m()` est ambigu. Il est nécessaire de préciser le point de vue, p.ex. `d.B::m()`
- Classe de base (**A**) virtuelle.
 - Une version de **A** pour **D**.
 - Invocation du constructeur de **A** depuis **D**, implicite (cst. par défaut) ou explicite: les invocations depuis ceux de **B** et **C** sont ignorées!



C++ 28

Exemple

```
class A {
    int a;
public:
    A(int x) : a(x) {
        cout << "A(" << a << ")\n";
    }
};

class B : virtual public A {
    int b;
public:
    B(int x) : b(x), A(x) {
        cout << "B(" << x << ")\n";
    }
};

class C : virtual public A {
    int c;
public:
    C(int x) : c(x), A(x) {
        cout << "C(" << x << ")\n";
    }
};

class D : public B, public C
{
public:
    D(int x, int y)
    : A(x + y), B(x), C(y) {
    }
};
```

Résultat de D d(1, 2);

- A(3)
- B(1)
- C(2)

Les appels aux constructeurs A() sont ignorés depuis celui de D()!

Généralement la classe de base (A) n'offrira qu'un constructeur sans paramètre.

C++

29

Pointeur unique (std::unique_ptr<T>)

- Un pointeur unique (`unique_ptr<T>`) est un pointeur intelligent qui est le seul à gérer une donnée et qui la supprime lorsqu'il n'est plus utilisé.
- Contrairement aux pointeurs partagés et faibles, il n'utilise pas de gestionnaire (il possède directement un pointeur sur la donnée). Le coût (performance et mémoire) d'un pointeur unique est donc plus faible.
- Son destructeur supprime directement la donnée si elle existe (rappel, l'opérateur `delete` ne fait rien sur un pointeur `nullptr`).
- L'unicité de `unique_ptr` est garantie par la suppression (= `delete`) du constructeur de copie et de l'opérateur d'affectation.
 - Les pointeurs uniques peuvent être passés par **déplacement** ou par **référence**, mais jamais par copie.
 - La donnée possédée par un pointeur unique peut être transférée *par déplacement* à un autre pointeur unique
 - Explicitement avec `std::move`
 - `p2 = std::move(p1);` // p2 possède la donnée, plus p1
 - Implicitement (eg variable locale `unique_ptr` rendue par une fonction)
 - `return p2;`

C++

3

- Les pointeurs uniques peuvent être créés par `std::make_unique`.
 - `unique_ptr<Person> p = std::make_unique<Person>("John");`

Pointeur unique

```
template <typename T>
class UniquePtr {
    T* ptr;
public:
    explicit UniquePtr(T* p = nullptr) : ptr(p) { cout << "Constructor\n"; }
    ~UniquePtr() { cout << "Destructor\n"; delete ptr; }
    // Suppression des opérateurs de copie
    UniquePtr(const UniquePtr&) = delete;
    UniquePtr& operator=(const UniquePtr&) = delete;

    T* operator->() const { return ptr; }
    T& operator*() const { return *ptr; }

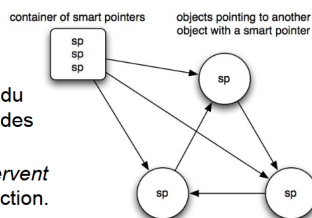
    UniquePtr(UniquePtr&& other) : ptr(std::exchange(other.ptr, nullptr)) { }
    UniquePtr& operator=(UniquePtr&& other) {
        if (this != &other) {
            delete ptr;
            ptr = std::exchange(other.ptr, nullptr);
        }
        return *this;
    }
};
```

C++

6

Pointeur partagé

- Un pointeur partagé (`shared_ptr`) est un pointeur intelligent permettant le partage d'une donnée entre différents pointeurs intelligents.
 - N'importe quel pointeur partagé référençant une donnée garantit qu'elle sera conservée.
 - Cette donnée est supprimée lorsque le dernier pointeur partagé cesse de la référencer.
- Limitation:
 - La formation d'un cycle empêche la destruction des données allouées.
 - P.ex., supprimer les pointeurs partagés du conteneur n'entraîne pas la destruction des trois objets qui se référencent en cycle.
 - Solution: utiliser des `weak_ptr` qui *observent* les données sans empêcher leur destruction.



C++

7

Opérations principales de shared_ptr

- Constructeur à partir d'un pointeur brut,
- Constructeur de copie (explicite),
- Destructeur,
- Surcharge de l'opérateur `=` : affectation (explicite) entre `shared_ptr`
- Surcharge de l'opérateur `*` : déréférencement de la donnée,
- Surcharge de l'opérateur `->` : accès à une propriété de la donnée,
- `T get()` : rendant le pointeur brut,
- `int useCount()` : rendant le nombre d'objets partageant la donnée,
- `bool unique`, rend vrai si aucun autre pointeur ne partage la donnée,
- `reset()` et `reset(p)` : réinitialisation (à vide ou sur un autre pointeur),
- Surcharge de l'opérateur de transtypage en `bool`: `true` si une donnée existe,
- Surcharge des opérateurs `==` et `!=` : test de l'égalité de contenu de pointeurs,
- Fonction `swap(p, q)` : échange du contenu de deux pointeurs.

C++

11

Pointeurs partagés : make_shared

- Quand un pointeur partagé est créé, deux allocations sont effectuées: une pour la donnée et l'autre pour l'objet gestionnaire.
- Pour pallier la lenteur induite par ces allocations la fonction générique `make_shared` effectue une seule allocation de taille suffisante pour contenir les deux objets.
 - `shared_ptr<Person> sp1(new Student(...));` // deux allocations
 - `shared_ptr<Person> sp2(make_shared<Student>(...));` // une seule allocation
- Fonctionne également avec des types "primitifs":
 - `shared_ptr<int> sp3 = make_shared<int>(42);`
- L'utilisation systématique de `make_shared` est recommandée car elle évite d'exposer des pointeurs bruts dans le code (pas de `new` explicite, pas de risque d'initialiser différents pointeurs partagés avec le même pointeur brut).

C++

12

Au delà de new/delete

- Les pointeurs intelligents peuvent être utilisés pour d'autres ressources que celles allouables par `new` / `delete` (à condition qu'elles sont représentables par un pointeur).
 - Gestion d'un tableau d'entiers (appel de `delete[]` automatique):
`std::unique_ptr<int[]> arr = std::make_unique<int[]>(10);`
`std::shared_ptr<int[]> sharedArr = std::make_shared<int[]>(10)`
 - Ouvrir et gérer automatiquement un fichier:
`void closeFile(FILE* fp) { if (fp) fclose(fp); }`
`std::unique_ptr<FILE, decltype(&closeFile)>`
`filePtr(fopen("example.txt", "w"), closeFile);`
 - Mémoire allouée via des API spécifiques (par exemple `malloc/free`):
`auto freeMemory = [](void* ptr) { free(ptr); };`
`std::unique_ptr<char, decltype(freeMemory)>`
`cString(static_cast<char*>(malloc(100)), freeMemory);`

C++

13

Pointeurs faibles

- Un pointeur faible (`weak_ptr`) est un pointeur intelligent qui référence une donnée sans empêcher sa suppression.
- Implémentation
 - L'objet gestionnaire possède deux compteurs indiquant le nombre de pointeurs intelligents référençant la donnée: un pour les pointeurs partagés et un pour les pointeurs faibles.
 - La donnée et son gestionnaire ne sont supprimés que lorsque ces deux compteurs valent 0.
 - Par contre, si seul le compteur de pointeurs partagés vaut 0, seule la donnée est supprimée, le gestionnaire existe toujours.
 - -> Un pointeur faible n'a pas l'assurance de référencer une donnée.
- La donnée (éventuellement) référencée par un pointeur faible ne s'obtient pas directement, il est nécessaire de passer d'abord par un pointeur partagé (constructeur ou méthode `lock`).

C++

14

Pointeurs *faibles* : exemples

■ Construction

```
shared_ptr<Person> sp = make_shared<Person>("John");
weak_ptr<Person> wp1(sp); // wp1 pointe sur John
weak_ptr<Person> wp2; // wp2 est vide
wp2 = sp; // wp2 pointe sur John
weak_ptr<Person> wp3(wp1) // wp3 pointe sur John
weak_ptr<Person> wp4; // wp4 est vide
wp4 = wp1; // wp4 pointe sur John
wp4.reset(); // wp4 est vide
```

■ Obtention d'un shared_ptr depuis un weak_ptr:

- `shared_ptr<Person> sp = wp.lock();`
if (sp) // opérateur bool
cout << sp->name() << endl;
- `shared_ptr<Person> sp(wp);` // lève `std::bad_weak_ptr` si vide

■ Tester si un weak_ptr est vide par wp.expired().

C++

15

Pointeur partagé sur this

- Avec des pointeurs bruts dans le corps d'une méthode d'une classe il est possible de passer l'objet courant en paramètre à une fonction ou une méthode (`o.m(*this)`), ou de le rendre comme résultat (`return *this`).

- En utilisant des pointeurs intelligents, créer un pointeur partagé sur `this` (`shared_ptr<Type>(this)`) est dangereux:

- Le pointeur partagé va désallouer l'objet courant alors que celui-ci n'a pas forcément été alloué dynamiquement.
- Si cet objet est déjà géré par un pointeur partagé, il sera libéré deux fois.

- Solution: faire hériter la classe devant obtenir un pointeur sur l'objet courant de la classe générique `enable_shared_from_this<T>`, définie dans l'en-tête `std::enable_shared_from_this`.

- Cette classe offre la méthode `shared_from_this()` rendant un pointeur partagé sur l'objet courant.

C++

16

Foncteurs (2)

```
class Sum
{
    int sum {0};
public:
    void operator()(int n) {
        sum += n;
    }
    int value() const {
        return sum;
    }
};

Array<int> array = { 1, 2, 3, 4, 5};
Sum s = for_each(array.begin(), array.end(), Sum());
// Sum() créé un nouvel objet temporaire, passé au for_each
cout << s.value() << endl;
```

C++

2

Expression lambda - exemple

```
double a = 2.14, b = 3.14, c = 42.0;
// Capturer automatiquement tous les objets par référence (a, b, et c)
auto fn1 = [&](double x, double y){return a * x + b * y + c;};
// Capturer automatiquement tous les objets par valeur (a, b, et c)
auto fn2 = [=](double x, double y){return a * x + b * y + c;};
// Lambda et types génériques:
// les types déduits pour x et y sont indépendants
auto f = [](auto x, auto y) { ... };
// x, et y doivent avoir le même type
auto g = [<class T>(T x, T y) {... };
f(1, 2);
g(1, 2);
f(1, 2.0);
g(1, 2.0); // Erreur de compilation
```

C++

8

Foncteur vs Lambda

- `Array<int> v = { 1, 2, 3, 4, 5};`

```
int sum = 0;
std::for_each(v.begin(), v.end(), [&sum](int x) {sum += x; });
cout << sum << endl;
```

// ou utiliser `std::accumulate`

```
int sum2 = std::accumulate(v.begin(), v.end(), 0,
    [](int x, int y) {return x+y; });
```

```
int sum3 = std::accumulate(v.begin(), v.end(), 0,
    std::plus<int>()); // <functional>
```

```
int sum4 = std::accumulate(v.begin(), v.end(), 0);
```

C++

13

Retourner un lambda

```
auto CreateAddLambda(int y) {
    return [y](int x) { return x + y; };
}

int main() {
    auto tenPlus = CreateAddLambda(10);
    cout << tenPlus(32) << endl;
}
```

Lambda récursif (C++23)

```
class A {
    int magical(int n)
    {
        auto factorial = [](int n, this auto&& self) {
            if (n <= 1) return 1;
            return n * self(n - 1);
        };
        return 42*factorial(n);
    }
};
```

this auto&& self (C++23) :
self désigne ici l'instance du foncteur généré par le lambda, et non l'objet A (représenté par le pointeur `this`). On peut donc écrire `self(n - 1)` pour faire un appel récursif.

Note: **self** n'est **pas** un mot clé, vous pouvez utiliser le nom de variable de votre choix.

C++

16

std::function vs lambda/foncteur

	Avantages	Inconvénients
<code>std::function<T></code> La signature de la fonction est de type T (explicitement représenté). Une fonction qui consomme un <code>std::function</code> n'a donc pas besoin d'être elle-même générique.	- Flexibilité : peut être lié à n'importe quel "callable" (lambda/foncteur, pointeur de fonction...) - Permet de stocker des fonctions pour une utilisation ultérieure (par exemple en temps qu'attribut d'une classe).	- Coût en performance et en mémoire (overhead)
lambda (ou foncteur) La signature de la fonction est implicite (le foncteur est représenté par un type générique). Une fonction qui consomme un foncteur/lambda doit donc elle-même être générique.	- Peuvent capturer des variables du "scope" - Performance optimale - Syntaxe concise avec les lambdas -	- Difficile à stocker pour une utilisation ultérieure (par exemple en temps qu'attribut d'une classe)

C++

20

```

struct X
{
    // prefix increment
    X& operator++()
    {
        // actual increment takes place here
        return *this; // return new value by reference
    }

    // postfix increment
    X operator++(int)
    {
        X old = *this; // copy old value
        operator++(); // prefix increment
        return old; // return old value
    }

    // prefix decrement
    X& operator--()
    {
        // actual decrement takes place here
        return *this; // return new value by reference
    }

    // postfix decrement
    X operator--(int)
    {
        X old = *this; // copy old value
        operator--(); // prefix decrement
        return old; // return old value
    }
};

```

